Assignment 1: Critique

Parva Shaileshkumar Patel (psp8502)

Collaboration and AI use statement

Setting Up & Understanding: I used Claude Code (Anthropic), an AI coding assistant, in VS Code. It helped me fix a build problem on my Mac, explained the codebase and the market-making concepts, and walked me message by message through the traces of all seven scenarios. I went through all seven scenarios, understood with the help of AI assistant. Moreover, to challenge my understanding, I gave quizzes using AI.
Critique: It helped me analyze my own analysis of the design decisions in the design document. It helped me spot what was missing and directed my using the hints to think deeply. I chose the decisions to experiment on.
Experiments: It wrote the code behind my measurements: benchmark scripts that run each scenario 30 times and summarise the results, a revised latency report that measures tick-to-trade and the two fixes I evaluate (risk checks that return a status value instead of throwing and a decision log written off the order path). It ran these experiments on my laptop and I reviewed the results.
Write-up: For the write-up, It helped me review the Grammar and any claim that I should not be inferring from the experiments. It reviewed and asked me to add one sentence to make my argument strong.
Collaboration: General discussion with batch-mates on the expectations and confirm that what I am doing is in-line with expectation.

Describe one possible execution of the hotpath.

One possible execution of hotpath is from the basic scenario. At 1,205,000 ns, a market data message reports a new 100 share bid at $100.01. ‘OnMarketData‘ copies the header out with ‘memcpy‘, checks the sequence number and decodes the AddOrder. The book finds the order’s slot in its array, adds 100 shares to the $100.01 level of the bid ladder and updates its cached best bid. ‘Requote‘ passes the risk checks and computes the microprice ($100.02) and the credit ($0.01). The bid is already at its target, but the ask must move from $100.04 to $100.03. So ‘SendQuote‘ appends to the quote history, writes a decision log line with ‘fprintf‘, builds a ‘ReplaceOrder‘ and calls ‘Send‘. This is one of the many hotpath and for this path no exception occurs.

Describe a flaw in the evaluation and a potential fix.

The flaw is that it reports one number which is the time from each message’s arrival to the end of its handler that is averaged over every delivery. It also includes timer ticks. In my understanding this is not tick-to-trade, which ends when an order leaves the application. The capture tap records that moment at every Send, but the report never uses it, so it also counts work done after an order has gone. Most deliveries never send an order. also mean hides the slow orders and is unstable as the same scenario’s average varied by up to 4.1 times between identical runs and in one run the first message alone made up 73% of it. The report should compute Send time minus arrival time for each order and give percentiles over runs, with no order deliveries and timer ticks reported separately.

Describe a good design decision in the market maker.

One of many good decisions is storing orders in an array indexed by the exchange’s order reference. Messages carry only that reference, so the lookup runs on almost every feed message. A lookup costs only a bounds check and one memory load. Because the exchange assigns references in sequence, the table is dense, and live orders sit close together in memory. A hash map would instead hash the key, follow a pointer to a allocated node, which could result in a cache miss, and allocate on every insert. A message that updates the book but sends nothing takes about one clock tick, including lookup. The drawback is the array’s fixed size. Reserving memory once at startup in exchange for a single memory load on nearly every feed message is the right trade for the hot path.

Describe a flaw in the market maker and a potential fix.

The risk checks failure by throwing exceptions and it argues that exceptions are free unless thrown and that a throw means we have left the hot path. In my understanding, that is not true. The checks fail routinely when a side of the book is empty, when a side is blocked by the position limit and after a halt. By my count the scenarios throw 73 times, against 58 order sending deliveries. A throw allocates memory and unwinds the stack which slows the system. When the position limit blocks the bid, the ask’s order is sent after the throw (1.8 µs, against 167 ns without exceptions) and a gap halt’s cancels are sent from the catch block (3.2 µs against 208 ns). In the fix, I replaced the exceptions with a [[*nodiscard*]] status value, so the compiler warns if a check’s result is ignored, which is the safety the design document wanted.

Describe a second flaw in the market maker and a potential fix.

Every order is written to the decision log with ‘fprintf‘ before it is sent. This is costly because text formatting, a file lock and an occasional write system call all fall inside each order’s tick-to-trade. The decision reason of keeping a record if the process dies mid-send does not hold. The ‘stdio‘ buffer isn’t flushed until it fills or program exit, so a crash loses recent lines anyway and durability would need an ‘fsync‘ per order. In the fix, the system now copies a fixed size entry into a preallocated buffer and write the text on the timer. The log is identical and the typical order fell time became one fourth. The order in the hot path above fell to one third. With both fixes combined, the median tick-to-trade across all scenarios fell to one fifth.